

Day 18 – PowerShell Detection Engineering

PowerShell Fundamentals

PowerShell is Microsoft's command-line shell and scripting language built on the .NET framework, widely used for system administration, automation, Active Directory management, cloud administration, and software deployment. Because it is installed by default on Windows systems and provides powerful automation capabilities, attackers frequently abuse it as a **Living Off the Land Binary (LOLBin)** to execute malicious actions without introducing additional tools.

Legitimate Administrator Use Cases

- System administration and automation
- Active Directory management
- Software installation and updates
- Windows configuration management
- Azure and Microsoft 365 administration
- Remote management and scripting

Common Attacker Abuse

Attackers leverage PowerShell throughout the attack lifecycle for:

- Executing **encoded commands** to evade detection
- Downloading malware using **Invoke-WebRequest**, **DownloadString**, or **Net.WebClient**
- Running fileless malware
- Disabling security controls
- Creating persistence through Scheduled Tasks or Registry modifications
- Command-and-Control (C2) communication
- Credential theft and lateral movement

Detection Engineering for PowerShell

High-quality PowerShell detections should focus on **command-line behavior**, **execution context**, and **multi-event correlation** rather than simply detecting powershell.exe.

Primary Detection Opportunities

Detection	Suspicious Behavior
Encoded PowerShell	Use of -EncodedCommand or Base64-encoded commands
Execution Policy Bypass	-ExecutionPolicy Bypass to ignore security restrictions
Hidden PowerShell	-WindowStyle Hidden or -NoProfile

Detection	Suspicious Behavior
Download Cradle	Invoke-WebRequest, DownloadString, Net.WebClient, or IEX
PowerShell with Network Activity	PowerShell followed by outbound HTTP/HTTPS communication

Primary telemetry includes **Windows Event ID 4688 (Process Creation)**, **PowerShell Event IDs 4103 (Module Logging) and 4104 (Script Block Logging)**, **Sysmon Event IDs 1 (Process Creation) and 3 (Network Connection)**, **Microsoft Defender XDR DeviceProcessEvents**, and **EDR process telemetry**.

Detection Tuning

Alerting on every PowerShell execution creates excessive false positives because administrators and enterprise management tools use PowerShell extensively. Effective tuning improves alert fidelity by focusing on suspicious behavior and execution context.

Recommended Tuning

- Prioritize **-EncodedCommand**, **-ExecutionPolicy Bypass**, **-NoProfile**, and **-WindowStyle Hidden**.
- Detect download cradles using **Invoke-WebRequest**, **DownloadString**, **Net.WebClient**, and **Invoke-Expression (IEX)**.
- Allowlist approved administrative scripts, management servers, and software deployment tools.
- Monitor PowerShell launched by Office applications, browsers, or immediately after RDP logins.
- Correlate PowerShell execution with network connections, Scheduled Tasks, Registry changes, and file creation before generating high-severity alerts.
- Enable **PowerShell Script Block Logging (Event ID 4104)** and **Module Logging (Event ID 4103)** for enhanced visibility.

Practical Exercise Findings

- Studied legitimate administrative uses of PowerShell and its abuse as a LOLBin for malware execution, fileless attacks, persistence, and command-and-control communication.
- Developed detection logic for **encoded commands**, **Execution Policy Bypass**, **download cradles**, **hidden PowerShell**, and **PowerShell with outbound network activity**, including telemetry sources, severity, MITRE ATT&CK mapping, and false-positive considerations.
- Tuned PowerShell detections by introducing command-line filtering, administrative allowlists, parent-process analysis, behavioral correlation, and Script Block Logging to significantly improve detection fidelity.

- Learned that **multi-stage correlation** (PowerShell → Download → Execution → Persistence → Network Communication) provides far higher confidence than isolated process execution alerts.
-

Investigation Scenario

The investigation correlated a complete attack chain beginning with a successful user logon, followed by **PowerShell execution using -EncodedCommand**, a malicious payload downloaded via **Invoke-WebRequest**, execution of the downloaded file, creation of a **Scheduled Task** for persistence, and an outbound **HTTPS** connection to an unknown external IP address. The combined evidence strongly indicated an active endpoint compromise involving malware delivery, execution, persistence, and command-and-control communication. The incident was classified as **Critical**, with recommendations to isolate the endpoint, terminate malicious processes, remove persistence mechanisms, preserve forensic evidence, reset potentially compromised credentials, and conduct environment-wide threat hunting for similar PowerShell activity, scheduled tasks, payload hashes, and network indicators.

MITRE ATT&CK

- **T1078** – Valid Accounts
 - **T1059.001** – PowerShell
 - **T1027** – Obfuscated Files or Information
 - **T1105** – Ingress Tool Transfer
 - **T1204** – User Execution (*if user-initiated payload execution*)
 - **T1053.005** – Scheduled Task/Job
 - **T1071.001** – Application Layer Protocol (Web Protocols)
-

Key Takeaway

PowerShell is an essential administrative tool but also one of the most frequently abused LOLBins in modern attacks. High-fidelity detections should prioritize suspicious command-line arguments, download cradles, encoded commands, and execution context while correlating PowerShell activity with authentication, persistence, and network telemetry. Continuous tuning, Script Block Logging, and multi-event correlation significantly reduce false positives and improve detection of sophisticated fileless and post-exploitation attacks.